

Machine Learning - Predicción de Salario

Angélica Ríos Cuentas - A01705651

Instituto Tecnológico de Estudios Superiores de Monterrey
Desarrollo de aplicaciones avanzadas de ciencias computacionales Gpo (302)

8 de Junio de 2026

Resumen

Este proyecto desarrolla un modelo de Machine Learning para predecir el salario anual de estudiantes recién egresados, expresado en LPA (*Lakhs Per Annum*), utilizando el dataset *Student Placement & Salary*. El proceso incluyó limpieza de datos, codificación de variables categóricas, normalización de variables numéricas y entrenamiento de modelos de regresión. Se implementó una regresión lineal múltiple como modelo base y posteriormente una red neuronal para mejorar la capacidad predictiva. El desempeño se evaluó mediante MAE y R2, además de comparar valores reales contra valores predichos mediante tablas y gráficas. Los resultados permitieron analizar el error del modelo y su capacidad para generalizar con datos no vistos.

Palabras clave: Machine Learning, regresión, predicción de salario, red neuronal, Random Forest.

1. Introducción

El uso de técnicas de Machine Learning permite analizar grandes volúmenes de datos y encontrar patrones que pueden ser utilizados para realizar predicciones. En este proyecto se plantea la construcción de un modelo capaz de estimar el salario anual de estudiantes recién egresados con base en sus características académicas, habilidades técnicas y condiciones laborales.

El dataset utilizado fue *Student Placement & Salary* [1]. Este conjunto de datos contiene información relacionada con estudiantes, sus calificaciones, habilidades, experiencia en proyectos, prácticas profesionales, tipo de empresa, rol laboral y salario estimado. Está compuesto por 20 columnas y 9000 registros. La variable objetivo del proyecto es “*salary_lpa*”, la cual representa el salario anual del estudiante en LPA (*Lakhs Per Annum*).

El objetivo principal del proyecto es desarrollar y evaluar un modelo de regresión capaz de predecir el salario de un estudiante a partir de sus características. Para ello, se aplicaron técnicas de preprocesamiento, normalización de datos, entrenamiento de modelos de

aprendizaje supervisado y evaluación con métricas de regresión. Además, se compararon diferentes enfoques para identificar cuál presentaba una mejor solución.

2. Dataset y analisis

2.1. Correlación

El análisis de correlación (Figura 1) permitió identificar las variables que presentan una relación más fuerte con el salario anual de los estudiantes. En particular, las variables relacionadas con habilidades técnicas muestran correlaciones positivas con “*salary_lpa*”, lo que indica que un mayor desarrollo de competencias puede estar asociado con mejores salarios. Asimismo, el tipo de compañía es una de las variables con mayor impacto, tanto positivo como negativo. Por ejemplo, trabajar en empresas multinacionales o tecnológicas de alto nivel se relaciona con salarios más altos, mientras que trabajar en startups presenta una relación negativa con el salario.

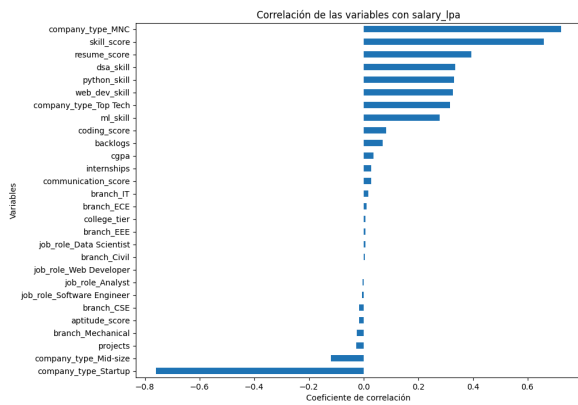


Figura 1. Tabla de correlación

Por otro lado, variables como el tipo de empleo y la rama de ingeniería no muestran una relación significativa con el salario dentro del conjunto de datos analizado.

3. Preparación y normalización de datos

Durante la etapa de preprocesamiento se identificó que el conjunto de datos presentaba un 1.4% de celdas vacías. Estos valores faltantes correspondían principalmente a registros de estudiantes que no obtuvieron empleo, es decir, aquellos casos en los que la variable “*placed*” tenía un valor igual a 0. Debido a que el objetivo del modelo es estimar el salario de estudiantes, se decidió eliminar dichas filas, ya que no aportaban información relevante para la predicción del salario y podían introducir ruido en el análisis.

Asimismo, se eliminó la columna “*placed*”, puesto que, después de remover los registros con valores faltantes, todos los estudiantes restantes presentaban un valor igual a 1 en dicha variable. Por lo tanto, esta columna dejó de aportar variabilidad al conjunto de datos y no fue considerada en el entrenamiento del modelo. Se eliminó la columna “*student_id*”, debido a que contiene identificadores únicos para cada estudiante. Este tipo de variable no representa una característica relevante, ya que no describe habilidades, desempeño académico ni condiciones laborales asociadas al salario.

Además, se descartaron las variables “*branch*” y “*job role*”, ya que en el análisis exploratorio no mostraron una relación significativa con “*salary_lpa*”. Se conservó la variable “*company_type*”, debido a que tipo de compañía tiene la correlación más alta y baja con el salario estimado. Esta variable categórica fue transformada mediante *One Hot Encoding*, usando *get_dummies* de Pandas, lo que permitió convertir sus categorías en columnas numéricas binarias [8].

Las variables independientes fueron separadas de la variable objetivo “*salary_lpa*” y se dividió el conjunto de datos en entrenamiento y prueba, utilizando 80% de los datos para entrenar el modelo y 20% para evaluarlo.

Finalmente, las variables numéricas fueron normalizadas mediante “*Standard Scaler*” [2]. Esta técnica transforma los valores para que tengan una media cercana a 0 y una desviación estándar cercana a 1. La normalización es importante porque evita que variables con escalas mayores tengan demasiada influencia en el entrenamiento del modelo.

Las variables numéricas normalizadas fueron: *cgpa*, *coding_score*, *communication_score*, *aptitude_score*, *projects* y *resume score*.

4. Modelo inicial

Como modelo inicial se implementó un modelo de regresión para predecir el salario anual de los estudiantes a partir de sus características académicas, técnicas y laborales.

El primer modelo permitió establecer una base de comparación para evaluar qué tan bien se podían aproximar los salarios utilizando las variables disponibles en el dataset. Posteriormente, se implementó una red neuronal de regresión con el objetivo de mejorar la capacidad predictiva del sistema.

La red neuronal fue construida con capas densas y funciones de activación ReLU en las capas ocultas porque ayuda a que la red aprenda relaciones más complejas entre las variables predictoras [3]. La capa de salida estuvo compuesta por una sola neurona, debido a que el problema corresponde a una tarea de regresión, es decir, la predicción de un valor numérico.

El modelo fue compilado utilizando el optimizador Adam, la función de pérdida MSE y la métrica MAE [6]. Se seleccionó MAE como una de las métricas principales porque permite interpretar el error promedio del modelo en la misma escala de la variable objetivo. Esto facilita comprender cuánto se desvía, en promedio, la predicción respecto al salario real.

4.1 Resultado del Modelo inicial

Para evaluar el desempeño del modelo se utilizaron métricas de regresión como el Error Absoluto Medio (MAE) y el coeficiente de determinación R2. Estas

métricas permiten analizar el nivel de error del modelo.

El MAE fue utilizado como métrica principal, ya que indica el error promedio absoluto entre los valores reales y los valores predichos. En este caso, al trabajar con “*salary_lpa*”, el MAE representa cuántas unidades de LPA se equivoca el modelo en promedio.

Además de las métricas numéricas, se generó una tabla comparativa entre los valores reales y los valores predichos [figura 2]. Esta tabla permite observar de manera individual el comportamiento del modelo en diferentes registros del conjunto de prueba, incluyendo el error de cada predicción.

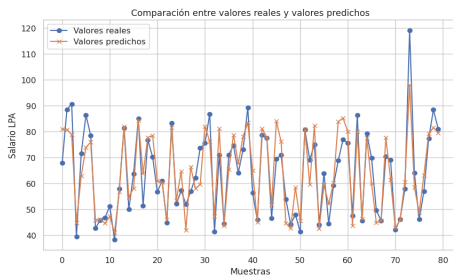


Figura 2 . Muestra de valores reales y valores predichos

También se elaboró una gráfica de todos los valores reales contra valores predichos [figura 3]. En esta gráfica, la línea diagonal representa la predicción ideal, donde el valor real coincide exactamente con el valor predicho. Se puede ver que el modelo si logra aproximar la tendencia del salario sin embargo cuenta con muchos valores sobre y bajo la línea de predicción.

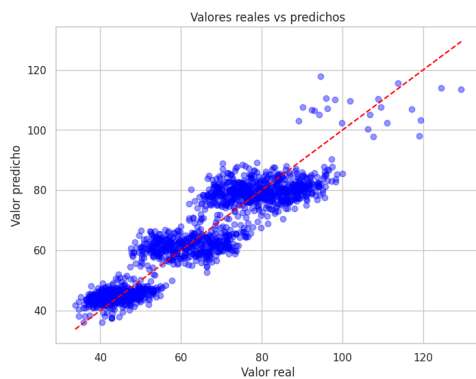


Figura 3 . Valores reales vs valores predichos

Por último, se analizó la evolución del MAE durante el entrenamiento. En la gráfica [figura 4] se observó que el MAE de entrenamiento disminuye progresivamente conforme avanzan las épocas, lo que indica que el modelo aprende de los datos de entrenamiento. Sin embargo, el MAE de validación se

mantiene más alto y no presenta una tendencia descendente. Este comportamiento sugiere que existe “overfitting”, ya que el modelo mejora en entrenamiento, pero no actúa de la misma manera con los datos no vistos.

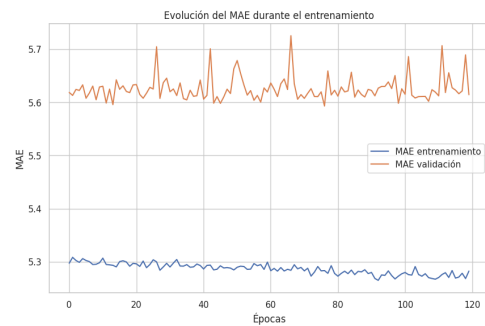


Figura 4. MAE durante el entrenamiento.

Con base en estos resultados, se considera necesario realizar ajustes de hiperparámetros, para ayudar a mejorar la generalización del modelo y reducir la diferencia entre el error de entrenamiento y el error de validación.

5. Modelo con ajustes

Para mejorar el modelo lo primero que se hizo fue una reestructuración en el split de los datos dividiendo 70% para train, 15% para validación y 15% para test. Con esto y un decremento de épocas en el modelo se logró obtener los resultados [Figura 5].

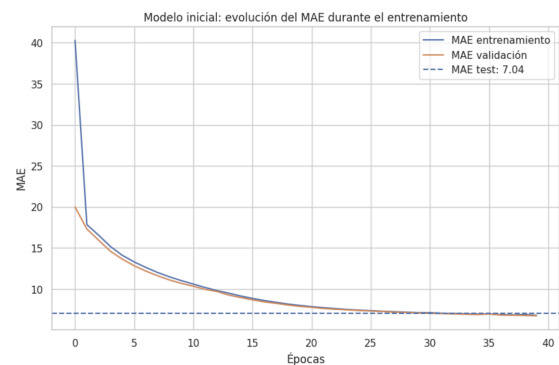


Figura 5. MAE durante el entrenamiento y test.

Aquí aunque el MAE incrementó se logró quitar el overfitting y tener un modelo con un fitting. Con él se buscaría disminuir el MAE pero mantener el fitting

Los ajustes realizados en el modelo tenían como objetivo mejorar la capacidad de aprendizaje y obtener mejores resultados en la predicción disminuyendo el

MAE. Primero, se incrementó la cantidad de épocas de 40 a 100, lo que significa que el modelo revisó los datos de entrenamiento más veces [6].

Después, se disminuyó el *batch size* de 100 a 40, es decir, el modelo utiliza grupos más pequeños de datos para actualizar los pesos. Esto ayudó a que el aprendizaje fuera más detallado, ya que no se esperaba a procesar una gran cantidad de registros antes de ajustar sus parámetros.

Por último, se agregó una capa *Dense* de 64 neuronas, la cual ayudó a mejorar la capacidad para identificar relaciones entre las variables de entrada.

5.1 Resultado del Modelo con Ajustes

Después de realizar los ajustes en la red neuronal, se observó una mejora en el comportamiento general del modelo. A diferencia del modelo inicial, donde existía una separación más marcada entre el error de entrenamiento y el error de validación, el modelo ajustado presentó una evolución más estable del MAE durante el entrenamiento. Esto indica que el modelo logró aprender de los datos sin memorizar excesivamente el conjunto de entrenamiento.

En la Figura 6 se muestra una comparación entre algunos valores reales, valores predichos y el error absoluto obtenido para cada caso. Esto demuestra que el modelo logra aproximarse a la tendencia general, pero todavía presenta limitaciones para predecir con alta precisión todos los casos individuales.

	Valor real	Valor predicho	Error absoluto
0	79.07	77.690002	1.38
1	49.36	42.139999	7.22
2	68.70	58.560001	10.14
3	59.41	59.310001	0.10
4	82.32	76.870003	5.45
5	63.24	65.519997	2.28
6	69.85	77.040001	7.19
7	60.82	60.130001	0.69
8	60.27	57.009998	3.26
9	57.59	60.380001	2.79
10	68.91	78.110001	9.20
11	86.49	76.459999	10.03
12	66.71	75.889999	9.18
13	72.15	79.419998	7.27
14	41.36	42.110001	0.75
15	74.33	72.309998	2.02
16	44.08	41.389999	2.69

Figura 6 . Valor real, valor predicho, error absoluto

La Figura 7 muestra la evolución del Error Absoluto Medio durante el entrenamiento del modelo ajustado [4]. En la gráfica se comparan el MAE de entrenamiento, el MAE de validación y el MAE obtenido en el conjunto de test. Al inicio del entrenamiento se observa un error elevado, sin

embargo, este disminuye de manera considerable durante las primeras épocas. Esto indica que el modelo comenzó a aprender rápidamente los patrones principales presentes en los datos.

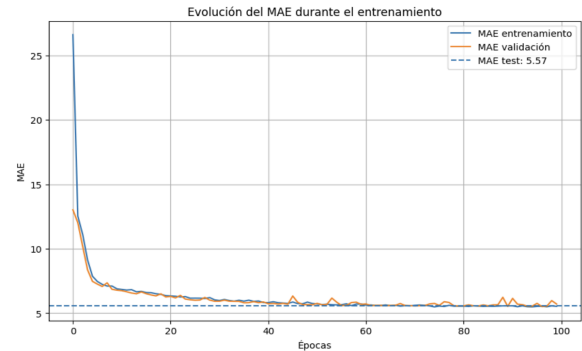


Figura 7. Evolución del MAE

Después de las primeras épocas, las curvas de entrenamiento y validación se estabilizan en valores cercanos entre sí. Este comportamiento es importante porque muestra que el modelo no solo está aprendiendo los datos de entrenamiento, sino que también mantiene un desempeño similar con datos de validación. La línea punteada representa el MAE obtenido en el conjunto de prueba, con un valor aproximado de 5.57 LPA. El resultado de 5.57 LPA en el conjunto de test se mantiene cercano a los valores finales de entrenamiento y validación, lo que indica que el modelo tiene una capacidad de generalización aceptable.

La Figura 8 muestra la relación entre los valores reales y los valores predichos. La línea diagonal representa la predicción ideal, es decir, el punto donde el valor predicho sería exactamente igual al valor real. En la gráfica se observa que la mayoría de los puntos siguen una tendencia cercana a la diagonal, lo cual indica que el modelo sí aprendió una relación entre las variables de entrada y el salario. Sin embargo, también se identifican puntos alejados de la línea, especialmente en salarios más altos, lo que sugiere que el modelo tiene mayor dificultad para estimar correctamente valores extremos [5].

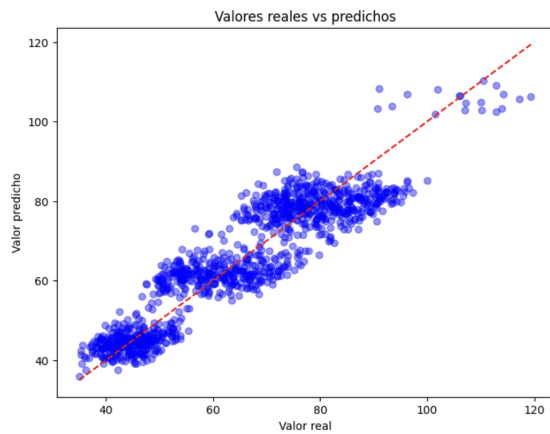


Figura 8. Valores reales vs valores predichos

El modelo ajustado obtuvo un MAE de prueba aproximado de 5.57 LPA. Esto significa que, en promedio, las predicciones del modelo se desvían alrededor de 5.57 LPA respecto al salario real. Aunque este resultado no elimina completamente el error, sí representa un modelo más equilibrado, ya que mantiene un comportamiento más estable entre entrenamiento, validación y prueba.

6. Random Forest

Como tercer propuesta se implementó un modelo Random Forest Regressor [7]. Este algoritmo trabaja mediante la combinación de varios árboles de decisión, donde cada árbol aprende diferentes patrones a partir de subconjuntos de datos y variables. Al combinar los resultados de muchos árboles, el modelo suele ser más estable que un solo árbol de decisión y puede capturar relaciones no lineales entre las variables predictoras y el salario.

Para este modelo se utilizaron 500 estimadores, una profundidad máxima de 10, selección de 5 características por división, “*min_samples_split*” igual a 3, “*min_samples_leaf*” igual a 1 y bootstrap activado. Estos hiperparámetros fueron definidos para controlar la complejidad del modelo y evitar que existiera un “*overfitting*”. A diferencia de la red neuronal, Random Forest no requiere entrenamiento por épocas ni normalización de datos para funcionar correctamente, aunque en este proyecto se mantuvo el mismo conjunto de variables para realizar una comparación justa entre los diferentes métodos.

El modelo Random Forest obtuvo un MAE de 5.35 LPA, un RMSE de 6.47 y un R2 de 0.8499 en el conjunto de prueba. Esto significa que, en promedio, el error absoluto del modelo fue ligeramente menor que el de la red neuronal mejorada. El valor de R2

indica que el modelo explica aproximadamente el 85% de la variabilidad del salario dentro del conjunto de prueba, lo cual representa un desempeño aceptable para un problema de regresión con variables académicas, técnicas y laborales.

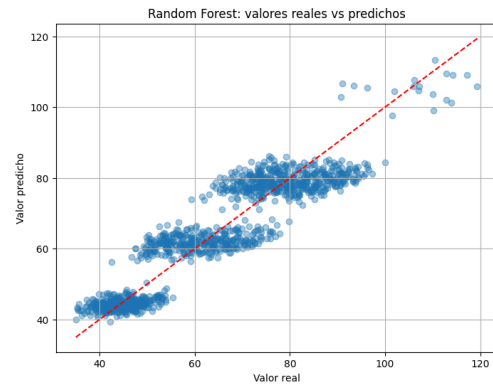


Figura 9. Random Forest: valores reales contra valores predichos.

En la Figura 7 se observa que las predicciones de Random Forest siguen una tendencia cercana a la línea diagonal ideal. Sin embargo, también se aprecia que el modelo tiene mayor dificultad para predecir algunos salarios altos, ya que varios puntos se alejan de la línea de predicción perfecta. Aun así, el comportamiento general es consistente y permite observar que Random Forest captura adecuadamente la relación entre las variables de entrada y el salario.

Al comparar los resultados, Random Forest fue el modelo con el MAE más bajo, mientras que la red neuronal ajustada presentó un comportamiento más estable durante el entrenamiento. Esto permite concluir que ambos enfoques son útiles, pero Random Forest resulta ligeramente más conveniente para este conjunto de datos debido a su menor error promedio y a su facilidad de interpretación en comparación con una red neuronal.

7. Conclusión

En este proyecto se desarrollaron y evaluaron diferentes modelos de Machine Learning para predecir el salario anual de estudiantes recién egresados. El proceso inició con la limpieza del conjunto de datos, la eliminación de variables que no aportaban información relevante, la transformación de variables categóricas mediante One Hot Encoding y la normalización de variables numéricas. Estas etapas fueron importantes porque permitieron preparar los datos de forma adecuada antes del entrenamiento de los modelos.

El modelo inicial permitió una aproximación al problema, pero presentó señales de overfitting, debido

a que el error de entrenamiento disminuía mientras que el error de validación se mantenía alto. Por esta razón, se realizaron ajustes en la arquitectura de la red neuronal, el número de épocas, el batch size y la separación de los datos en entrenamiento, validación y prueba. Con estos cambios, el modelo logró un comportamiento más constante y equilibrado.

Posteriormente, se implementó Random Forest Regressor como modelo alternativo. Este modelo obtuvo el menor MAE del proyecto, con un error promedio aproximado de 5.35 LPA y un R2 de aproximadamente 0.85. Estos resultados muestran que el modelo puede utilizarse como una herramienta para estimar salarios con base en características académicas, habilidades y tipo de compañía.

En general, el proyecto demuestra que las técnicas de Machine Learning pueden ser útiles para identificar patrones. Sin embargo, también se reconoce que las predicciones no son siempre correctas y que el salario puede depender de muchos otros factores externos que no se encuentran en el dataset, como la ubicación, demanda, experiencia real, habilidades blandas y condiciones específicas de cada empresa y empleo.

Como trabajo futuro, se recomienda probar con la combinación de otros datasets, y la búsqueda de otras técnicas o algoritmos que disminuyan aún más el MAE.

Referencias

- [1] Mishra, A. (2026). *Student Placement & Salary Dataset (Skills based)* [Data set]. Kaggle. <https://www.kaggle.com/datasets/amaymishra11/student-placement-and-salary-dataset-skills-based>
- [2] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [3] TensorFlow. (2024). El modelo secuencial. TensorFlow. https://www.tensorflow.org/guide/keras/sequential_model?hl=es-419
- [4] Matplotlib Development Team. (2026). *Multiple lines using pyplot*. Matplotlib 3.10.9 documentation. https://matplotlib.org/stable/gallery/pyplots/pyplot_three.html
- [5] Matplotlib Development Team. (2026). *scatter(x, y)*. Matplotlib 3.10.9 documentation. https://matplotlib.org/stable/plot_types/basic/scatter_plot.html
- [6] TensorFlow. (s. f.). *Training & evaluation with the built-in methods*. TensorFlow. https://www.tensorflow.org/guide/keras/training_with_built_in_methods
- [7] Scikit-learn. (s. f.). *RandomForestRegressor*. Scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
- [8] The pandas development team. (2026). *pandas.get_dummies*. pandas documentation. https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html